

Computer Architecture and Organization

Chapter 2: Data Representation

Decimal to Binary Conversion – Fixed Point Numbers

Converting the Fractional Part —The Multiplication Method

A binary fraction is represented in the general form:

$$b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + b_{-3} \times 2^{-3} + \dots$$

If we multiply the fraction by 2, then we will obtain:

$$b_{-1} + b_{-2} \times 2^{-1} + b_{-3} \times 2^{-2} + \dots$$

If we iterate this process on the remaining fraction, then we will obtain successive b_i .

Decimal to Binary Conversion – Fixed Point Numbers

Remainder Method:

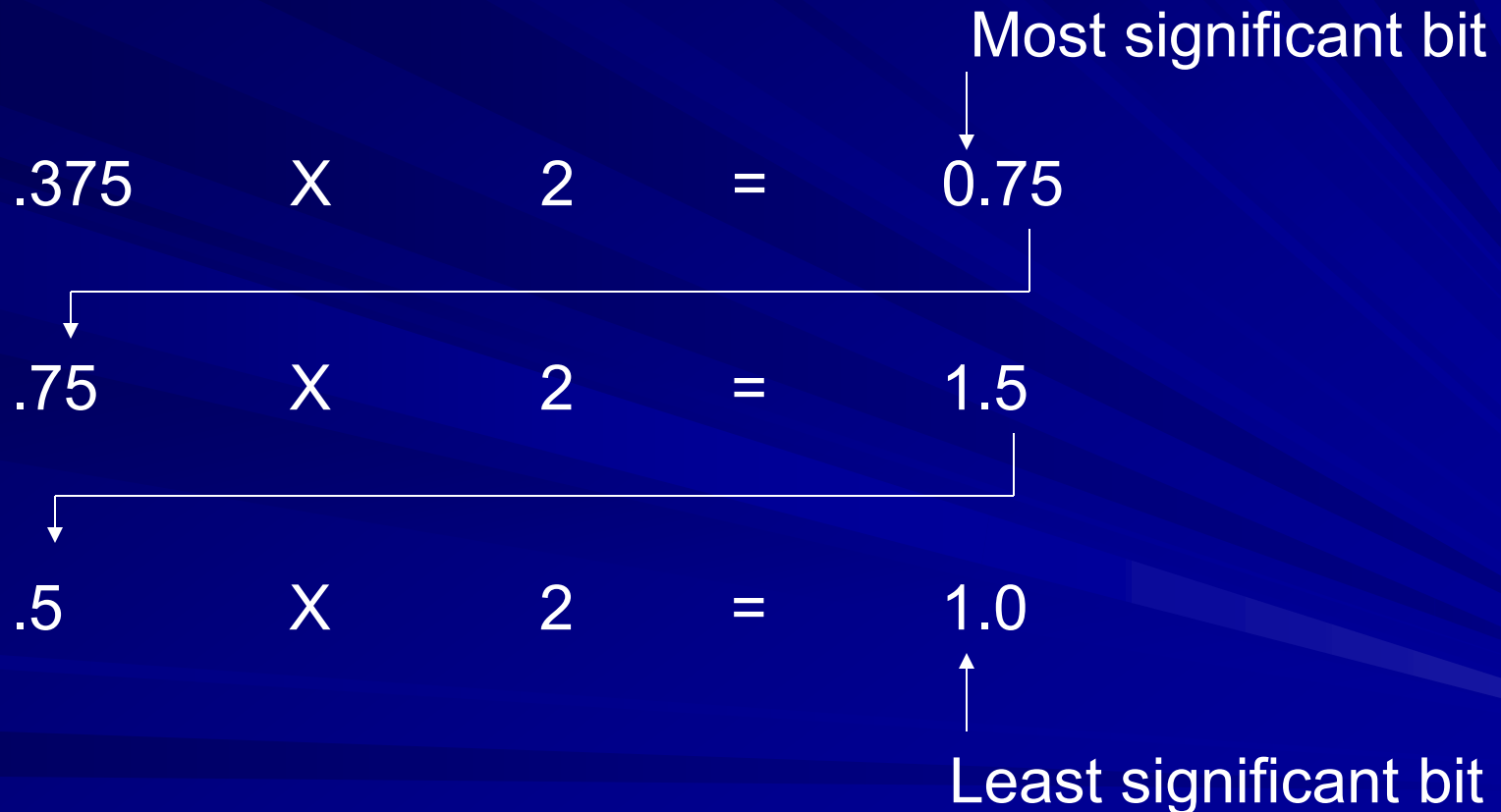
Conversion of the decimal number 1234 to binary by successive halving, starting at the top and working downward. For example, 77 divided by 2 yields a quotient of 38 and a remainder of 1

Quotient	Remainder	
1234		
617	0	
308	1	
154	0	
77	0	
38	1	
19	0	
9	1	
4	1	
2	0	
1	0	
0	1	
		1 0 0 1 1 0 1 0 0 1 0

Therefore, $1234_{10} = 10011010010_2$

Decimal to Binary Conversion – Fixed Point Numbers

Converting the Fractional Part — The Multiplication Method



$$(0.375)_{10} = (.011)_2$$

Binary to Decimal Conversion

Conversion of the binary number 101011011 to decimal by

101011011

1	$\times 2^0 =$	1
1	$\times 2^1 =$	2
0	$\times 2^2 =$	0
1	$\times 2^3 =$	8
1	$\times 2^4 =$	16
0	$\times 2^5 =$	0
1	$\times 2^6 =$	64
0	$\times 2^7 =$	0
1	$\times 2^8 =$	<u>256</u>

347₁₀

Therefore, $101011011_2 = 347_{10}$

Binary, Octal, and Hexadecimal Radix Representations

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Representing Negative Numbers

- Signed magnitude
- One's complement

Note that: Both Signed magnitude and one's complement have two representation for zero: a plus zero and a minus zero

- Two's complement
- Excess $2^m - 1$ – Examples: for an eight bit number, 0 becomes $128 + 0 = 128$, 1 becomes $1 + 128 = 129$, -5 becomes $-5 + 128 = 123$, and so on (binary format).

Negative Binary Numbers – 8 bits

N decimal	N binary	-N signed mag.	-N 1's compl.	-N 2's compl.	-N excess 128
1	00000001	10000001	11111110	11111111	01111111
2	00000010	10000010	11111101	11111110	01111110
3	00000011	10000011	11111100	11111101	01111101
4	00000100	10000100	11111011	11111100	01111100
5	00000101	10000101	11111010	11111011	01111011
6	00000110	10000110	11111001	11111010	01111010
7	00000111	10000111	11111000	11111001	01111001
8	00001000	10001000	11110111	11111000	01111000
9	00001001	10001001	11110110	11110111	01110111
10	00001010	10001010	11110101	11110110	01110110

Negative Binary Numbers – 8 bits

N Decimal	N Binary	-N Signed Magnitude	-N 1's Complement	-N 2's Complement	-N excess 128
20	00010100	10010100	11101011	11101100	01101100
30	00011110	10011110	11100001	11100010	01100010
40	00101000	10101000	11010111	11011000	01011000
50	00110010	10110010	11001101	11001110	01001110
60	00111100	10111100	11000011	11000100	01000100
70	01000110	11000110	10111001	10111010	00111010
80	01010000	11010000	10101111	10110000	00110000
90	01011010	11011010	10100101	10100110	00100110
100	01100100	11100100	10011011	10011100	00011100
127	01111111	11111111	10000000	10000001	00000001
128	Nonexistent	Nonexistent	Nonexistent	10000000	00000000

Signed Magnitude

In signed magnitude, the **Most Significant Bit (MSB)** acts strictly as a sign flag.

- **Rule:** Set the MSB to 0 for positive numbers and 1 for negative numbers. The remaining bits represent the absolute value (magnitude) in standard unsigned binary.
- **Example -5:** The magnitude of 5 in 7 bits is 0000101. Adding the negative sign bit (1) at the front yields 10000101.
- **The Zero Problem:** It produces two bit-patterns for zero:
 - $+0 = 00000000$
 - $-0 = 10000000$

One's Complement

One's Complement

One's complement represents negative values by **inverting every single bit** of the corresponding positive number.

- **Rule:** Keep positive numbers in standard binary. To make a number negative, flip all 0s to 1s and all 1s to 0s.
- **Example -5:** Start with +5 (00000101). Flip every bit to get 11111010.
- **The Zero Problem:** Like signed magnitude, it suffers from a dual-zero anomaly:
 - $+0 = 00000000$
 - $-0 = 11111111$

Two's Complement

Two's complement is the standard system used in modern CPUs because it **eliminates duplicate zeros** and allows identical hardware to handle both addition and subtraction.

- **Rule:** Find the one's complement of the number, then add 1 to the lowest position.
- **Example -5:**
 1. Start with +5 (00000101).
 2. Flip the bits to get the one's complement (11111010).
 3. Add 1: $11111010 + 1 = 11111011$
- **The Zero Fix:** Adding 1 to the one's complement of zero ($11111111 + 1$) causes a carry bit to drop off the left edge, cleanly rolling the system back over to 00000000

Excess $-2^{(m-1)}$ (Offset Binary)

Excess notation shifts the value scale by adding a constant **bias** value to the true number before converting it into unsigned binary. For an m -bit integer, the standard bias is 2^{m-1} .

- **Rule for $m=8$, Bias = 128:** Calculate Value + 128, then convert that unsigned sum directly into binary.
- **Example 0:** $0 + 128 = 128$ or 10000000
- **Example 1:** $1 + 128 = 129$ or 10000001
- **Example -5:** $-5 + 128 = 123$ or 01111011

Summary of Representations

The mathematical bit patterns for decimal -5 in an 8-bit architecture alter drastically depending on the chosen format:

- Signed Magnitude: **10000101**
- One's Complement: **11111010**
- Two's Complement: **11111011**
- Excess-128: **01111011**

Binary Arithmetic

Addend	0	0	1	1
Augend	<u>+0</u>	<u>+1</u>	<u>+0</u>	<u>+1</u>
Sum	0	1	1	0
Carry	0	0	0	1

The addition table in binary

Decimal	1's complement	2's complement
10	00001010	00001010
+ (-3)	<u>11111100</u>	<u>11111101</u>
+7	1 00000110	1 00000111
	 carry 1	 discarded
	<u>00000111</u>	

Addition in one's complement and two's complement.

Binary Arithmetic

- Addition
 - 1's Complement
 - 2's Complement
- Subtraction
 - Use addition – add the complement of the subtrahend to the minuend
 - Both are complement form, just add them together.
- Overflow
 - If the addend and augend are of opposite signs, overflow error cannot occur. If they are of the same sign and the result is of the opposite sign, overflow error has occurred.
 - In both one's and two's complement, overflow occurs if and only if the carry into the sign bit differs from the carry out of the sign bit.

Example (4-bit system):

- $13 + 5$ **OR** $1101 + 0101 = 10010$ (which is 2 with a carry of 1).
- **Because a 5th bit is required, the 4-bit space overflows.**

Example: An 8-bit signed integer can only hold up to 127.

- If you have 127 and add 1, it overflows and becomes -128 instead of 128.

Subtraction of Binary Numbers

Binary subtraction takes away one value from another, borrowing a 1 from the next column on the left when subtracting 1 from 0

Rules:

- $0 - 0 = 0$
- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ with a borrow of 1

Example ($11_2 - 01_2$, or $3_{10} - 1_{10}$)

1 1

- 0 1

1 0 (Result is 2 in decimal)

Binary Multiplication

- Binary multiplication uses partial products shifted to the left, which are then added together.

1 0

× 1 1

Rules:

- 0 times 0 = 0
- 0 times 1 = 0
- 1 times 0 = 0
- 1 times 1 = 1

1 0 (10 × 1)

1 0 (10 × 1, shifted left)

1 1 0 (Result is 6 in decimal)

Ex. (10_2 times 11_2 , or 2_{10} times 3_{10})

Binary Division

- Binary division works just like long division in decimal math, using repeated subtraction and bringing down bits from left to right.

Ex. ($110_2 \div 10_2$, or $6_{10} \div 2_{10}$):

$$\begin{array}{r} 11 \text{ (Quotient = 3 in decimal)} \\ \text{-----} \\ 10 \overline{) 110} \\ \underline{- 10} \\ \text{-----} \\ 10 \\ \underline{- 10} \\ \text{-----} \\ 00 \end{array}$$

Floating Point Numbers

$$n = \pm f \times 10^e$$

Diagram illustrating the components of a floating point number n :

- Sign**: Indicated by the $\pm$ symbol.
- fraction or mantissa**: Indicated by the f .
- Radix**: Indicated by the 10 .
- Exponent**: Indicated by the e .

Floating-Point Numbers

$$3.14 = 0.314 \times 10_1 = 3.14 \times 100$$

$$0.000001 = 0.1 \times 10^{-5} = 1.0 \times 10^{-6}$$

$$1941 = 0.1941 \times 10^4 = 1.941 \times 10^3$$

a number representation of the Floating Point
Formula shown in the previous slide:

- A signed three-digit fraction range in magnitude from $0.1 \leq |f| < 1$ or zero and a signed two-digit exponent has the negative number range:
from -0.100×10^{-99} to $-0.999 \times 10^{+99}$ and
positive number range
from $+0.100 \times 10^{-99}$ to $+0.999 \times 10^{+99}$.
A span of nearly 199 order of magnitude.

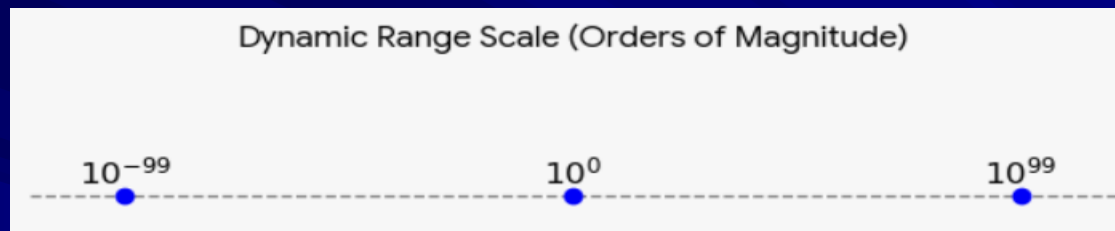
Floating-Point Numbers

- **Overflow** – the magnitude is too large to be represented by its positive exponent value (for both + and - numbers)
- **Underflow** – the magnitude is too small to be represented by its negative exponent value (for both + and - numbers)
- In many cases the underflow error is less serious than the overflow error.
- **Fraction (mantissa)** represent the density between two real numbers. Increasing the fraction increases the accuracy.
- **Exponent** represent the size (scope) of the number. Increasing the exponent increase the size of regions 2 and 6.
- The spacing between adjacent expressible numbers is not constant but the relative error introduced by **rounding** is the same for small number as large ones.

Floating-Point Numbers

- A signed three-digit fraction range in magnitude from $0.1 \leq |f| < 1$ or zero and a signed two-digit exponent has the negative number range: from -0.100×10^{-99} to $-0.999 \times 10^{+99}$ and positive number range from $+0.100 \times 10^{-99}$ to $+0.999 \times 10^{+99}$.

A span of 198 order of magnitude.



- Underflow Boundary: The smallest positive non-zero value is $+0.100 \times 10^{-99}$. Numbers closer to zero cannot be accurately represented.
- Overflow Boundary: The largest positive value is $+0.999 \times 10^{99}$. Numbers larger than this cause system errors.
- The negative scale mirrors these limits exactly on the opposite side of zero.

Calculate Order Span

- The total span measures the breadth of the system's dynamic range. We calculate it by finding the distance between the maximum and minimum powers:

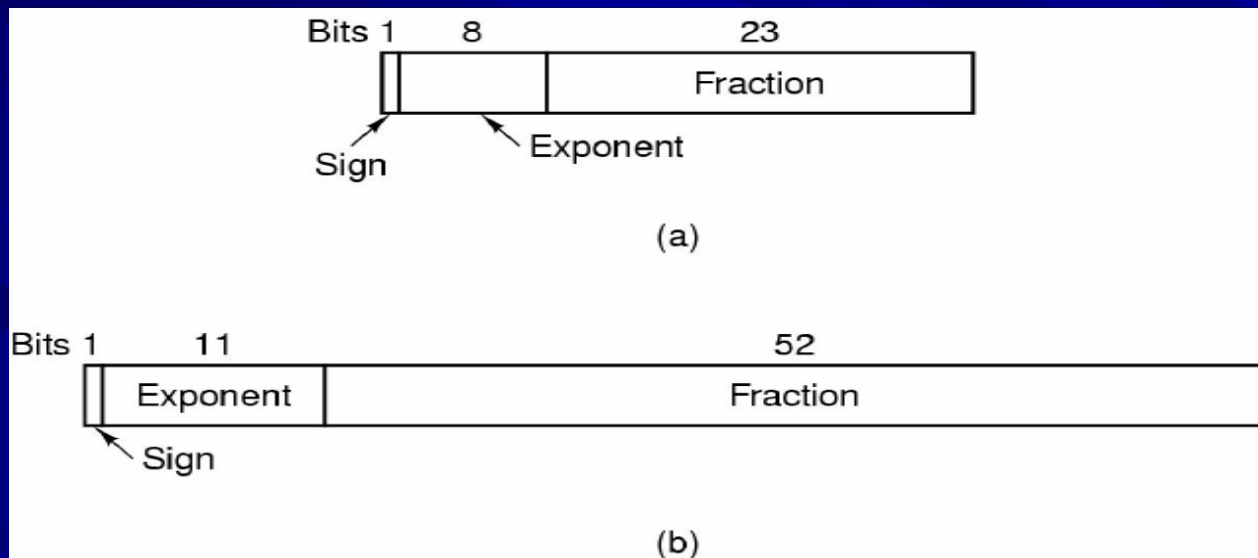
$$\text{Span} = 99 - (-99) = 198$$

This power difference allows a span of nearly 199 orders of magnitude.

IEEE 754 Floating-Point

$$(-1)^{\text{Sign}} \times (1.\text{Fraction}) \times 2^{\text{Exponent}-\text{Bias}}$$

- Single precision – 32 bits
- Double precision – 64 bits
- Extended precision – 80 bits
- Starting from the leftmost bits
 - Sign; Exponent, 8 for single (excess 127) and 11 for double (excess 1023); Fraction, 23 for single and 52 for double.
- Fraction – implied 1 followed by 23 or 52 explicit bits



IEEE 754 Floating-Point

Item	Single precision	Double precision
Bits in sign	1	1
Bits in exponent	8	11
Bits in fraction	23	52
Bits, total	32	64
Exponent system	Excess 127	Excess 1023
Exponent range	-126 to +127	-1022 to +1023
Smallest normalized number	2^{-126}	2^{-1022}
Largest normalized number	approx. 2^{128}	approx. 2^{1024}
Decimal range	approx. 10^{-38} to 10^{38}	approx. 10^{-308} to 10^{308}
Smallest denormalized number	approx. 10^{-45}	approx. 10^{-324}

Floating Point Operations

- Addition/subtraction
 - Align the exponent
 - Add (subtract) the mantissa
 - Normalize the result
- Multiplication (division)
 - Multiply (divide) the mantissa
 - Add the exponent (subtract)
 - Normalize the result

How to represent a Number in IEEE 754 32-bit Floating Point Notation

■ Example: Given: $(263.3)_{10}$

Required: represent the given number in IEEE754 (single precision)

Solution:

a. convert to binary

2[263	
2[131	r. 1
2[65	r. 1
2[32	r. 1
2[16	r. 0
2[8	r. 0
2[4	r. 0
2[2	r. 0
2[1	r. 0
2[0	r. 1

.3 x 2 = 0.6	] 0
.6 x 2 = 1.2	] 1
.2 x 2 = 0.4	] 0
.4 x 2 = 0.8	] 0
.8 x 2 = 1.6	] 1
.6 x 2 = 1.2	] 1

$(263)_{10} = (100000111)_2$

$(0.3)_{10} = (01001100\dots)_2$

$(263.3)_{10} = (100000111.0100110011\dots)_2$

continuation

- Represent binary form to Scientific notation

1.000001110100110011... $\times 2^8$

- Write the Scientific notation in IEEE 754

In 32 bit FPN – exponential bias is 127

Note:

Sign bit (0 or 1)	exp bit	fraction bits (mantissa)
0	$127 + 8 = 135$ $(135)_{10} = (10000111)_2$	000001110100110011...

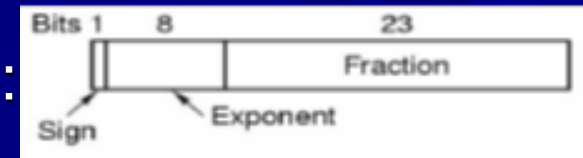
8 is added to the bias since its positive (if negative exp, subtract)

Therefore, final IEEE 754 representation: (should be 32 bits so add 0000
0100 0011 1000 0011 1010 0110 0110 0110 0000)

IEEE 754 32-bit FPN to Decimal Equivalent

- Given: 0100 0011 0101 0100 0000 0000 0000 0000
- Required: FPN to Decimal Equivalent
- Solution:

Notes:



0100 0011 0101 0100 0000 0000 0000 0000

- 0 means positive decimal
- Exponent: $(10000110)_2 \rightarrow 2^1 + 2^2 + 2^7 = 134$
Exponential bias = 127 $\rightarrow 134 - 127 = 7$
- $m = 1 \times 2^{-1} + 1 \times 2^{-3} + 1 \times 2^{-5} + \dots$
 $= 0.65625$
- Decimal Equivalent: $(-1)^{\text{sign}} \times (1+m) \times 2^e$
 $= (-1)^0 \times (1+0.65625) \times 2^7 = 1.65625 \times 2^7$
 $= 212$

Scientific Notation:

$1.\text{mmmm} \times 2^e$

References

- William Stallings, Computer Architecture and Organization. 11th Edition, 2019.
- Structured Computer Organization by Andrew Tannenbaum